Birla Institute of Technology & Science, Pilani

Work Integrated Learning Programmes Division

AIML* ZG523 · MLOps

Lab Experiment 1

Experiment Tracking and Model Registry with MLflow

Module / Session	M2 Model Experimentation and Packaging · Contact Session 4
Mapped Outcomes	LO1, LO2 (CO1, CO2)
Mode	Self-paced, offline-capable (internet needed only for one-time package install)
Estimated Effort	2.5 – 3 hours
Readings	T2 Ch. 6 (Model Development & Offline Evaluation); T1 Ch. 4

1. Lab Storyline — Welcome to CartVista

CartVista is a fast-growing online grocery platform. You have just joined its three-person Data Science team, which owns one business-critical model: predicting which customers are likely to churn (stop ordering) in the next 30 days, so that the retention team can intervene with targeted offers.

On your first day, you discover how the team currently works. Priya trains models in a Jupyter notebook and saves them as final_model.pkl, final_model_v2.pkl and final_model_v2_REALLY_FINAL.pkl. Arjun keeps his best hyperparameters in a WhatsApp message to himself. Nobody can answer the questions the retention team keeps asking:

- “Which model version is actually running right now, and how good is it?”
- “Last month's model felt better. Can we go back to it?”
- “Priya's AUC was 0.84, Arjun's was 0.86 — were they even measured on the same test data?”

Your team lead gives you your first assignment: introduce experiment tracking and a model registry using MLflow, so that every training run is recorded, comparable and reproducible — and so there is exactly one, unambiguous “champion” model that downstream systems can load by name.

The bigger picture: The churn model you track today is the same model you will containerize in Lab 2, serve as a microservice in Lab 3, and monitor for drift in Lab 4. Work cleanly — future-you is your first customer.

2. Objective

To set up MLflow-based experiment tracking and a model registry for CartVista's churn-prediction model: log training runs with their parameters, metrics and artifacts; compare candidate models systematically; and register and alias the best model so it can be retrieved programmatically by downstream consumers.

3. Learning Outcomes (Summarized)

After completing this lab, you will be able to:

- Structure an ML training script so that every run is tracked — parameters, metrics, artifacts and the model itself.
- Use the MLflow UI to compare runs and justify a model choice with evidence rather than memory.
- Register a model in the MLflow Model Registry, manage versions, and assign a “champion” alias.
- Load a registered model by name and alias from a separate consumer program — decoupling training from usage.
- Explain why tracking and registry are the foundation on which deployment (Lab 2–3) and monitoring (Lab 4) are built.

4. Prerequisites and One-Time Setup

You need Python 3.10+ and a terminal. Everything runs locally; no cloud account and no dataset download is required — the lab generates its own synthetic data. The only step requiring the internet is the package installation below. Do it once; the rest of the lab works fully offline.

4.1 Create the project and install packages (one-time, online)

```
  terminal
mkdir cartvista-mlops && cd cartvista-mlops
python -m venv .venv

# Activate the environment:
#   Windows:        .venv\\Scripts\\activate
#   macOS / Linux:  source .venv/bin/activate

pip install mlflow==2.14.1 scikit-learn==1.5.0 pandas numpy
```

Pinned versions keep the lab reproducible. Newer versions generally work; if outputs differ slightly, that is fine.

4.2 Verify

terminal
```
python -c "import mlflow, sklearn; print('MLflow', mlflow.__version__, '| sklearn',
sklearn.__version__)"
MLflow 2.14.1 | sklearn 1.5.0
```

Expected output (green blocks throughout this manual show expected output).

Windows note: If you see a UnicodeDecodeError anywhere in this lab, set the environment variable PYTHONUTF8=1 (PowerShell: $env:PYTHONUTF8="1") and retry.

5. Step-by-Step Instructions

Create each file exactly as shown, inside the cartvista-mlops folder. Run the commands from that folder with your virtual environment activated. Each step ends with the expected outcome — check it before moving on.

Step 1 — Generate the CartVista dataset (offline)

Real churn data is confidential, so CartVista's data engineering team has given you a generator that produces a statistically realistic snapshot of 5,000 customers. The churn signal is deliberately embedded (long-tenured, frequent, discount-using customers churn less; customers raising many support tickets churn more) so your models have something genuine to learn.

generate_data.py
```
"""CartVista Lab 1 -- synthetic customer churn snapshot (fully offline)."""
import numpy as np
import pandas as pd

def generate(n=5000, seed=42):
    rng = np.random.default_rng(seed)
    tenure_months    = rng.integers(1, 72, n)          # how long a customer
    monthly_spend    = np.round(rng.uniform(200, 3000, n), 2)
    orders_per_month = rng.poisson(3, n)
    support_tickets  = rng.poisson(1.2, n)
    uses_discounts   = rng.integers(0, 2, n)           # 1 = redeems coupons
    app_sessions     = rng.poisson(12, n)              # sessions last month

    # Hidden 'true' churn tendency (students: do NOT use this in features!)
    logit = (-1.2
             - 0.030 * tenure_months
             + 0.0004 * monthly_spend
             - 0.25  * orders_per_month
             + 0.45  * support_tickets
             - 0.40  * uses_discounts
             - 0.05  * app_sessions)
    p_churn = 1.0 / (1.0 + np.exp(-logit))
    churn = (rng.uniform(0, 1, n) < p_churn).astype(int)

    return pd.DataFrame({
        "tenure_months": tenure_months,
        "monthly_spend": monthly_spend,
        "orders_per_month": orders_per_month,
        "support_tickets": support_tickets,
        "uses_discounts": uses_discounts,
        "app_sessions": app_sessions,
        "churn": churn,
    })

if __name__ == "__main__":
    df = generate()
    df.to_csv("churn_data.csv", index=False)
    print(f"Wrote churn_data.csv | rows={len(df)} | churn rate={df.churn.mean():.2%}")
```

terminal
```
python generate_data.py
Wrote churn_data.csv | rows=5000 | churn rate=8.42%
```

Expected outcome: churn_data.csv exists; churn rate around 8–10% (an imbalanced problem, like real churn). Your exact rate may differ by a few tenths.

Step 2 — Feel the pain: train without tracking

First, work the way Priya and Arjun do today. This script trains one model and prints its scores — and remembers nothing.

train_no_tracking.py

```
"""The BEFORE picture: training with no memory of what was done."""
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score, accuracy_score

df = pd.read_csv("churn_data.csv")
X, y = df.drop(columns=["churn"]), df["churn"]
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

model = RandomForestClassifier(n_estimators=50, max_depth=4, random_state=42)
model.fit(X_tr, y_tr)

auc = roc_auc_score(y_te, model.predict_proba(X_te)[:, 1])
acc = accuracy_score(y_te, model.predict(X_te))
print(f"AUC={auc:.4f}  accuracy={acc:.4f}")
```

terminal

```
python train_no_tracking.py
AUC=0.7482  accuracy=0.9152
```

Now run it again after changing n_estimators to 200 and max_depth to 8. Better? Worse? After five more edits you will not remember which combination produced which number — and neither will your team. That gap between "ran an experiment" and "can reconstruct an experiment" is exactly what MLflow closes.

Checkpoint question: Why is accuracy 91% while AUC is only ~0.75? Hint: what would a model that predicts "no churn" for everyone score on each metric, given the churn rate from Step 1? Write your answer down — it is part of your deliverable.

Step 3 — Add MLflow tracking to the training script

Now the AFTER picture. The tracked version accepts the model type and hyperparameters from the command line, and logs everything to a local SQLite-backed MLflow store — parameters, metrics, the trained model, and the exact data snapshot's fingerprint.

train_tracked.py

```
"""CartVista churn training WITH MLflow experiment tracking."""
import argparse, hashlib, pathlib
import pandas as pd
import mlflow, mlflow.sklearn
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score, accuracy_score, f1_score

TRACKING_URI = "sqlite:///mlflow.db"     # local file DB -> registry works offline
EXPERIMENT   = "cartvista-churn"

def build_model(kind, args):
    if kind == "logreg":
        return LogisticRegression(C=args.C, max_iter=1000)
    if kind == "rf":
        return RandomForestClassifier(n_estimators=args.n_estimators,
                                      max_depth=args.max_depth, random_state=42)
    raise ValueError(f"unknown model kind: {kind}")

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--model", choices=["logreg", "rf"], required=True)
    ap.add_argument("--C", type=float, default=1.0)              # logreg
    ap.add_argument("--n-estimators", type=int, default=100)     # rf
    ap.add_argument("--max-depth", type=int, default=6)          # rf
    args = ap.parse_args()
```

```
    mlflow.set_tracking_uri(TRACKING_URI)
    mlflow.set_experiment(EXPERIMENT)

    df = pd.read_csv("churn_data.csv")
    X, y = df.drop(columns=["churn"]), df["churn"]
    X_tr, X_te, y_tr, y_te = train_test_split(
        X, y, test_size=0.25, random_state=42, stratify=y)

    with mlflow.start_run(run_name=f"{args.model}-run"):
        # ---- 1. log WHAT we are about to do ----
        mlflow.log_param("model_kind", args.model)
        if args.model == "logreg":
            mlflow.log_param("C", args.C)
        else:
            mlflow.log_params({"n_estimators": args.n_estimators,
                               "max_depth": args.max_depth})
        data_md5 = hashlib.md5(
            pathlib.Path("churn_data.csv").read_bytes()).hexdigest()[:10]
        mlflow.log_param("data_fingerprint", data_md5)

        # ---- 2. do it ----
        model = build_model(args.model, args)
        model.fit(X_tr, y_tr)

        # ---- 3. log HOW WELL it went ----
        proba = model.predict_proba(X_te)[:, 1]
        preds = model.predict(X_te)
        mlflow.log_metrics({
            "test_auc": roc_auc_score(y_te, proba),
            "test_accuracy": accuracy_score(y_te, preds),
            "test_f1": f1_score(y_te, preds),
        })

        # ---- 4. log THE MODEL ITSELF ----
        mlflow.sklearn.log_model(model, artifact_path="model")

        run = mlflow.active_run().info
        print(f"Logged run {run.run_id[:8]}  auc={roc_auc_score(y_te, proba):.4f}")

if __name__ == "__main__":
    main()
```

Read the four numbered comment blocks — they are the whole mental model of tracking: log what you are doing, do it, log how well it went, log the resulting artifact. Every tracked training script you ever write is a variation of this shape.

Step 4 — Run a small experiment sweep

Run six candidate configurations — three logistic regressions and three random forests:

```
  terminal
python train_tracked.py --model logreg --C 0.1
python train_tracked.py --model logreg --C 1.0
python train_tracked.py --model logreg --C 10.0
python train_tracked.py --model rf --n-estimators 50  --max-depth 4
python train_tracked.py --model rf --n-estimators 200 --max-depth 8
python train_tracked.py --model rf --n-estimators 400 --max-depth 12
Logged run 3fa8c1d2  auc=0.7311
Logged run 91b0e4f7  auc=0.7335
Logged run c44d90aa  auc=0.7338
Logged run 7e2f5b19  auc=0.7482
Logged run ab6602cd  auc=0.7561
Logged run 5d19f3e8  auc=0.7549
```

Expected outcome: six runs logged; run IDs will differ, AUC values should be within ±0.01 of these. Note the mid-sized forest beating the largest one — memorable evidence that bigger is not automatically better.

Step 5 — Compare runs in the MLflow UI

```
  terminal
```

```
mlflow ui --backend-store-uri sqlite:///mlflow.db --port 5000
```

Open http://127.0.0.1:5000 in your browser and click the cartvista-churn experiment. You should see all six runs. Now do the three things the retention team could never do before:

- Sort by test_auc descending. The best run should be an rf run — click it and confirm its exact hyperparameters.
- Tick all six runs and press Compare. Study the parallel-coordinates view: how does max_depth relate to test_auc?
- Open any run's Artifacts tab. The trained model is stored there with its environment specification — this is what makes the run reproducible, not just recorded.

Expected outcome: A table of six runs with parameters and three metrics each; the top test_auc around 0.75–0.76; every run carrying the same data_fingerprint value (same data — so the comparison is fair, answering Arjun-vs-Priya once and for all).

Step 6 — Register the best model and crown a champion

Tracking answers “what did we try?”. The registry answers a different question: “what is THE model right now?”. This script finds the best run by test AUC, registers it under a single well-known name, and points the alias champion at it.

register_model.py

```
"""Pick the best tracked run and make it the registered champion."""
import mlflow
from mlflow import MlflowClient

mlflow.set_tracking_uri("sqlite:///mlflow.db")
MODEL_NAME = "cartvista-churn"

client = mlflow.MlflowClient()
exp = client.get_experiment_by_name("cartvista-churn")
best = client.search_runs(
    [exp.experiment_id],
    order_by=["metrics.test_auc DESC"],
    max_results=1)[0]

print(f"Best run {best.info.run_id[:8]} auc={best.data.metrics['test_auc']:.4f} "
      f"params={best.data.params}")

mv = mlflow.register_model(
    model_uri=f"runs:/{best.info.run_id}/model",
    name=MODEL_NAME)

client.set_registered_model_alias(MODEL_NAME, "champion", mv.version)
print(f"Registered {MODEL_NAME} v{mv.version} and set alias @champion")
```

terminal

```
python register_model.py
Best run ab6602cd auc=0.7561 params={'model_kind': 'rf', 'n_estimators': '200',
  'max_depth': '8', 'data_fingerprint': '9f1c2ab3e4'}
Registered cartvista-churn v1 and set alias @champion
```

Expected outcome: the Models tab of the MLflow UI now shows cartvista-churn, Version 1, alias @champion. Re-running this script after new experiments would create v2, v3, … and move the alias — versions are immutable, the alias is a movable pointer.

Step 7 — Consume the champion from a separate program

Finally, prove the decoupling. This consumer knows nothing about training, hyperparameters or run IDs — it asks the registry for “the champion of cartvista-churn” by name. This is exactly how your Lab 3 microservice will load the model.

consume_model.py

```
"""A downstream consumer: load the champion by alias and score customers."""
import mlflow, pandas as pd

mlflow.set_tracking_uri("sqlite:///mlflow.db")
model = mlflow.pyfunc.load_model("models:/cartvista-churn@champion")
```

```
batch = pd.read_csv("churn_data.csv").drop(columns=["churn"]).head(5)
scores = model.predict(batch)
for i, s in enumerate(scores):
    print(f"customer_{i:03d} -> churn prediction: {int(s)}")
```

```
terminal
python consume_model.py
customer_000 -> churn prediction: 0
customer_001 -> churn prediction: 0
customer_002 -> churn prediction: 0
customer_003 -> churn prediction: 0
customer_004 -> churn prediction: 1
```

Expected outcome: five predictions with no training code imported. Your 0/1 pattern may differ slightly. If you later retrain and re-point @champion, this consumer picks up the new model with zero code changes — hold that thought until Lab 3.

6. What You Have Built — the Map

Artifact	Role in CartVista	Reappears in
`churn_data.csv`	Versioned data snapshot (fingerprinted in every run)	Labs 2–4 (same data)
`mlflow.db + mlruns/`	Team memory: every experiment, comparable and reproducible	Lab 2 packaging pulls from here
`cartvista-churn @champion`	The single source of truth for “the model”	Lab 3 serves it; Lab 4 monitors it

7. Exercises

Complete E1–E4 (required); E5 is a stretch goal. Record answers and screenshots in your submission document.

E1 — Close the metric gap (10 min)

In Step 2's checkpoint you explained the accuracy-vs-AUC gap. Now act on it: add precision and recall to the metrics logged in train_tracked.py and re-run the best configuration. Which of the two is more important for CartVista's retention team, whose offers cost money to send? One paragraph.

E2 — A third challenger (15 min)

Extend build_model() with kind “gb” (sklearn's GradientBoostingClassifier, expose learning_rate and n_estimators). Run two configurations. Does the champion change when you re-run register_model.py? Screenshot the Models page showing the version history.

E3 — Tags for humans (10 min)

Add mlflow.set_tag("author", "<your-name>") and a “purpose” tag to each run. Then, in the UI, filter runs to show only yours. Why are tags the right home for this rather than parameters?

E4 — Break it, observe, explain (10 min)

Delete one row from churn_data.csv (keep a backup!) and run one more experiment. Compare its data_fingerprint with earlier runs in the UI. In two sentences: what real Priya-vs-Arjun dispute does this one logged parameter prevent? Restore the backup afterwards.

E5 (stretch) — Autolog vs manual (20 min)

Create train_auto.py using mlflow.sklearn.autolog() and no manual log calls. Compare what it captures against your manual runs. List one thing autolog captured that you did not, and one thing your manual script logs that autolog cannot know.

8. Troubleshooting (Offline First-Aid)

Symptom	Fix
`mlflow: command not found`	Virtual environment not activated — re-run the activate command from Step 4.1.
Port 5000 already in use	Use --port 5001 (macOS: AirPlay Receiver squats on 5000 — disable it or change port).
Registry error mentioning "file store"	You launched the UI without --backend-store-uri sqlite:///mlflow.db, or a script omitted set_tracking_uri. The registry needs the SQLite backend — check both.
Runs invisible in UI	UI and scripts must point at the same store and be started from the same folder. Restart the UI from cartvista-mlops with the exact Step 5 command.
`UnicodeDecodeError (Windows)`	Set PYTHONUTF8=1 in the same terminal session, then retry.
AUC differs from manual by > 0.01	Check random_state=42 in both the split and the forest, and that you regenerated data with the default seed.

9. Deliverables Checklist (for EC-1 Lab Assignment)

Submit a single PDF containing:

- Screenshot: experiment table with all runs, sorted by test_auc (Step 5).
- Screenshot: Models page showing cartvista-churn versions and the @champion alias (after E2).
- Terminal output of consume_model.py (Step 7).
- Written answers: Step 2 checkpoint, E1, E3, E4 (and E5 if attempted).
- One reflection paragraph: "What breaks in this setup when Priya and Arjun each run MLflow on their own laptops?" — we pick this thread up in Session 6.

Academic integrity: Your run IDs, timestamps, data fingerprint and author tags are unique to your machine and are checked. Submissions sharing identical run IDs will be treated as copied.